# Sensores de PyBullet

## Análisis de los Sensores

Clase de Robótica

April 18, 2026

# Visión Artificial: La Cámara en PyBullet

A diferencia de un sensor ultrasónico o un láser (que usan el motor de físicas y colisiones para calcular choques), la cámara en PyBullet utiliza el **Motor de Renderizado** (OpenGL o TinyRenderer).

En la vida real, un lente captura luz. En simulación, la cámara "dibuja" la escena 3D desde un punto de vista específico. Para lograr esto sin que el procesador colapse, PyBullet nos pide definir matemáticamente dos cosas: **desde dónde miramos** y **cómo es nuestro lente**.

# Las Dos Matrices Mágicas

Para tomar una foto, necesitamos configurar dos conceptos fundamentales de la computación gráfica. No se preocupen, PyBullet tiene funciones que resuelven la matemática por nosotros:

- **Matriz de Vista (View Matrix):** Define la **Posición** y **Orientación** de la cámara en el mundo. Físicamente representa: ¿Dónde pusimos el trípode y hacia qué punto exacto estamos mirando?
- **Matriz de Proyección (Projection Matrix):** Define la **Óptica del lente**. ¿Es un lente gran angular? ¿Es cuadrada o rectangular? ¿Hasta cuántos metros de distancia puede ver con claridad?

# 1. La Matriz de Vista (¿Dónde y hacia dónde?)

PyBullet usa la función `computeViewMatrix`. Esta necesita 3 coordenadas espaciales $(X, Y, Z)$:

- **Posición (`cameraEyePosition`):** La coordenada exacta donde ubicamos el lente del robot.
- **Objetivo (`cameraTargetPosition`):** La coordenada hacia donde apunta el lente (Por ejemplo, un obstáculo o un microplástico en la arena).
- **Vector Arriba (`cameraUpVector`):** Le dice al motor qué dirección es "arriba" para que la foto no salga de cabeza. Casi siempre es $(0, 0, 1)$ apuntando hacia el cielo en el eje Z.

# Código: Configurando la Matriz de Vista

```python
import pybullet as p

# 1. Donde ponemos la camara (ej: a 1 metro de altura)
pos_camara = [0.0, 0.0, 1.0]

# 2. Hacia donde mira (ej: un metro hacia adelante en X)
punto_objetivo = [1.0, 0.0, 0.0]

# 3. Direccion 'Arriba' (Eje Z positivo)
vector_arriba = [0, 0, 1]

# PyBullet calcula la Matriz de Vista
matriz_vista = p.computeViewMatrix(
    cameraEyePosition=pos_camara,
    cameraTargetPosition=punto_objetivo,
    cameraUpVector=vector_arriba
)
```

# 2. La Matriz de Proyección (La óptica del Lente)

Ahora usamos la función `computeProjectionMatrixFOV` para configurar el comportamiento del lente:

- **FOV (Field of View):** Campo de visión en grados. El ojo humano tiene unos 120°, una cámara web normal ronda los 60°.
- **Aspect Ratio (Relación de Aspecto):** Proporción ancho/alto. Si nuestra foto es cuadrada (ej. 256x256), el valor es 1.0. Si es panorámica (1920x1080), es $\approx 1.77$.
- **Near y Far Plane:** La distancia mínima y máxima a la que la cámara dibuja objetos. Ayuda a ahorrar memoria no procesando objetos infinitamente lejanos.

# Código: Configurando la Óptica

```
# Configuracion de nuestro "lente virtual"
fov = 60.0                  # 60 grados de apertura
aspect = 1.0                # Foto cuadrada (1:1)
near_val = 0.1              # No ver nada mas cerca de 10 cm
far_val = 100.0             # No renderizar nada mas alla de 100 metros

# PyBullet calcula la Matriz de Proyeccion
matriz_proyeccion = p.computeProjectionMatrixFOV(
    fov=fov,
    aspect=aspect,
    nearVal=near_val,
    farVal=far_val
)
```

# ¡Click! Tomando la Foto

Con ambas matrices preparadas en la memoria, le ordenamos a PyBullet que renderice la imagen usando el comando principal: `getCameraImage`.

```
ancho = 256   # Pixeles de ancho
alto = 256    # Pixeles de alto

# Disparamos la captura
datos_camara = p.getCameraImage(
    width=ancho,
    height=alto,
    viewMatrix=matriz_vista,
    projectionMatrix=matriz_proyeccion
)
```

# ¿Qué nos devuelve `getCameraImage`?

PyBullet es muy generoso y nos entrega mucha más información que solo colores. Nos devuelve una tupla con **5 elementos**. ¡Es vital saber identificar qué es cada uno para no provocar errores en nuestro código!

- **Índice 0 y 1:** Ancho y Alto de la imagen (enteros).
- **Índice 2: Imagen de Color RGBA** (El que usaremos hoy).
- **Índice 3: Mapa de Profundidad** (Una escala de grises que nos dice qué tan lejos está físicamente cada píxel).
- **Índice 4: Máscara de Segmentación** (A cada objeto se le asigna un ID único. PyBullet nos pinta la imagen separando matemáticamente cada objeto).

# El Problema de la "Lista Plana" (1D vs 3D)

**¡Presten mucha atención a esto!** La imagen de color (Índice 2) que nos da PyBullet **no viene como una foto normal**. Por rendimiento interno, PyBullet nos entrega un **arreglo lineal (1D)** gigantesco: `[R, G, B, A, R, G, B, A...]`.

Librerías como OpenCV esperan una **matriz 3D** (Ancho $\times$ Alto $\times$ Canales). Si le pasamos la lista de PyBullet directo a OpenCV o a una Red Neuronal, el programa colapsará inmediatamente.

# El Rescate: Procesando los Datos con Numpy

Para transformar esa tira infinita de números en una imagen procesable, usamos `numpy.reshape`.

```
import numpy as np
import cv2  # OpenCV para visualizacion

# 1. Extraemos el arreglo lineal gigante de colores
rgb_plano = datos_camara[2]

# 2. Reformateamos (Reshape) a Matriz 3D (Alto, Ancho, 4
    Canales RGBA)
imagen_rgba = np.reshape(rgb_plano, (alto, ancho, 4))

# 3. OpenCV trabaja en formato BGR, asi que convertimos:
imagen_bgr = cv2.cvtColor(imagen_rgba, cv2.COLOR_RGBA2BGR)

# Mostrar en pantalla
cv2.imshow("Camara del Robot", imagen_bgr)
cv2.waitKey(1)
```

# Integración: ¿Cómo agregamos la cámara al URDF?

**La Pregunta Trampa:** En PyBullet, a diferencia de Gazebo o ROS, los sensores de visión **no se simulan directamente dentro del archivo URDF** con etiquetas `<sensor>`.

En PyBullet, hacemos un truco dinámico:

1. En el URDF, creamos un **Eslabón vacío** (un Link sin masa ni forma) en el frente del robot. Lo bautizamos como `camera_link`.
2. En nuestro bucle `while` de Python, preguntamos constantemente: *"¿En qué coordenada* $(X, Y, Z)$ *exacta está el* `camera_link` *ahora mismo?"*
3. PyBullet nos da esa coordenada, y nosotros la usamos para actualizar y mover nuestra **Matriz de Vista**. ¡Así la cámara viaja pegada al robot!

# Código: Montando la Cámara en el Robot

Usamos `getLinkState` para actualizar la cámara en cada paso de simulación.

```
# Dentro de tu bucle principal (while True):

# 1. Obtener posicion del eslabon de la camara (supongamos indice 3)
estado_link = p.getLinkState(id_robot, 3)
posicion_link = estado_link[0] # Tupla (X,Y,Z) actual

# 2. Actualizar nuestra Matriz de Vista con la nueva posicion
matriz_vista_dinamica = p.computeViewMatrix(
    cameraEyePosition=posicion_link,      # Camara viaja en el eslabon
    cameraTargetPosition=punto_frontal,   # Apuntando al frente
    cameraUpVector=[0, 0, 1]
)

# 3. Tomar foto (getCameraImage) usando esta nueva matriz...
```

# El Problema: Relatividad Espacial

Para crear nuestra Matriz de Vista, queremos anclar la cámara en la "nariz" del robot, digamos a 0.2 metros hacia adelante de su centro.

**¿Por qué no podemos simplemente hacer:** `X_camara = X_robot + 0.2`**?**
Si el robot está mirando directo por el eje X, esa suma funciona. Pero, ¿qué pasa si el robot gira 90° hacia la izquierda? Su "adelante" ahora es el eje Y. Si seguimos sumando en X, la cámara quedará flotando mágicamente a un costado de la rueda derecha.

Sumar posiciones directamente solo funciona si el robot nunca rota.

# Coordenadas Locales vs Globales

Para resolver esto, debemos entender dos conceptos:

- **Coordenadas Globales:** El centro del universo en PyBullet $(0, 0, 0)$. Nunca cambian.
- **Coordenadas Locales:** El centro del robot. Desde el punto de vista del robot, su nariz **siempre** está a 0.2 metros adelante, sin importar hacia dónde mire.

Necesitamos una operación matemática que tome la coordenada *Local* de la nariz, considere cómo está rotado el robot, y la traduzca a la coordenada *Global* del mundo para poner la cámara allí.

# La Solución: `multiplyTransforms`

En álgebra lineal, esto se resuelve multiplicando matrices de transformación homogénea. PyBullet nos ahorra esa matemática pesada con la función `multiplyTransforms`.

**¿Qué recibe la función?**

1. La Posición y Orientación de nuestro **Sistema Base** (El chasis del robot).
2. La Posición y Orientación **Local** del punto que buscamos (Ej. $[0.2, 0.0, 0.1]$).

**¿Qué nos devuelve?**
La coordenada $(X, Y, Z)$ global y exacta de ese punto, calculada en tiempo real según la inclinación, rotación y posición actual del robot.

# Ejemplo: Calculando la Posición de la Cámara

Así es como lo aplicamos en nuestro bucle de simulación:

```
# 1. Obtenemos posicion y rotacion (cuaternion) del robot
pos_robot, ori_robot = p.getBasePositionAndOrientation(
    id_robot)

# 2. Definimos donde queremos la camara LOCALMENTE
# (0.2m adelante en X, y 0.1m arriba en Z)
pos_local = [0.2, 0.0, 0.1]
ori_local = [0, 0, 0, 1] # Sin rotacion adicional

# 3. PyBullet hace la magia matematica
pos_camara, _ = p.multiplyTransforms(
    pos_robot, ori_robot,
    pos_local, ori_local
)

# pos_camara ahora tiene el (X,Y,Z) global perfecto
```

# El Panel Lateral: PyBullet al Descubierto

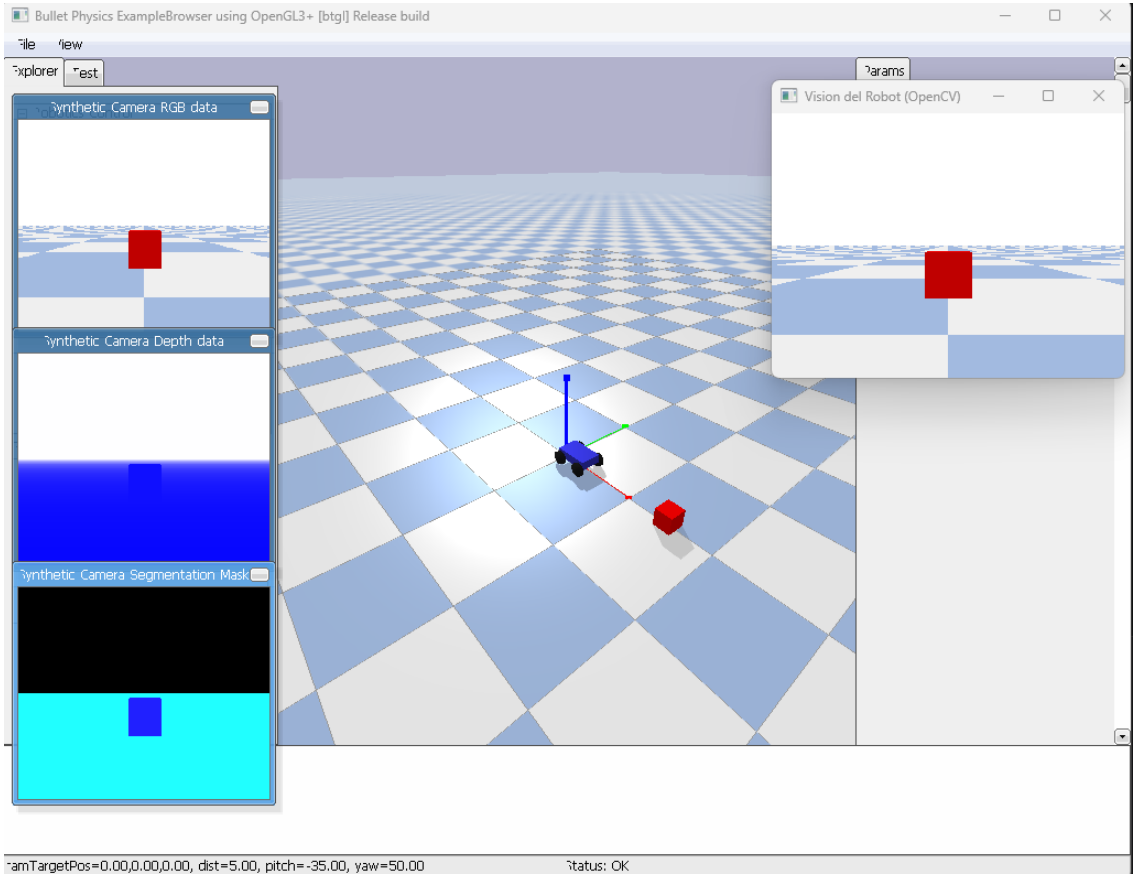


Figure: Panel Lateral de PyBullet

# El Panel Lateral: PyBullet al Descubierto

¿Recuerdan que `getCameraImage` nos devuelve una tupla con 5 elementos?

PyBullet es tan amable (o entrometido) que por defecto abre **tres ventanas en su panel lateral** de la interfaz gráfica. Estas ventanas nos permiten ver en tiempo real exactamente qué está procesando el motor de renderizado antes de pasarnos los datos al código en Python.

# 1. Synthetic Camera RGB data

Esta ventana nos muestra la **imagen de color normal**.

Es la representación visual exacta de la **matriz plana gigante (1D)** que discutimos anteriormente. Es esta misma información la que extraemos (en el Índice 2 de la tupla), procesamos matemáticamente con Numpy para pasarla a formato 3D, y finalmente enviamos a nuestra propia ventana de visualización de OpenCV.

# 2. Synthetic Camera Depth data (Profundidad)

Representa el **Mapa de Profundidad**. En esta imagen, los colores no representan luz, sino **distancia**.

Si observan con detención:

- **Zonas claras (blancas):** Objetos muy cercanos al lente del robot.
- **Zonas oscuras (azules/negras):** Objetos lejanos, acercándose al horizonte de simulación.

Gracias a esta matriz (Índice 3), nuestro algoritmo puede saber exactamente a cuántos metros de distancia se encuentra cada píxel individual.

# 3. Synthetic Camera Segmentation Mask

Es la **Máscara de Segmentación**. Aquí, el motor le asigna un color plano e identificador (ID) único a cada modelo 3D cargado en el mundo físico.

**¿Por qué es vital para la Inteligencia Artificial?**
En el mundo real, reconocer un objeto depende de la luz, reflejos o sombras. En la simulación, esta máscara le dice a la IA con 100% de precisión qué píxeles exactos son "Cubo", cuáles son "Piso" y cuáles "Cielo", facilitando el entrenamiento de redes neuronales (Índice 4).

# Pro-Tip: Limpiando la Pantalla

Estas ventanas son geniales para entender qué ve el robot, pero **consumen recursos de video** y quitan espacio valioso en la pantalla.

Si prefieren ver solo el mundo 3D interactivo y la ventana flotante de OpenCV que programamos, pueden apagar todo este panel lateral agregando una simple línea de código justo después de conectarse con `p.connect(p.GUI)`:

```
# Oculta los paneles laterales y ventanas de depuracion
p.configureDebugVisualizer(p.COV_ENABLE_GUI, 0)
```

# Extrayendo Profundidad: El Índice 3

Si queremos que el robot sepa a qué distancia está un obstáculo, el color no nos sirve. Necesitamos la **Matriz de Profundidad**, que PyBullet nos entrega en el **Índice 3** de `getCameraImage`.

Igual que con la imagen de color, PyBullet nos entrega una lista lineal infinita que debemos reformatear usando Numpy a una matriz de 2D (Alto $\times$ Ancho).

**El problema:** Si leemos el valor de un píxel de esa matriz, nos daremos cuenta de que **no está en metros**. Nos entregará valores como `0.9998` o `0.9951`. ¿Por qué?

# El Z-Buffer y la Escala no Lineal

En computación gráfica, la profundidad se calcula usando algo llamado **Z-Buffer**. Por eficiencia matemática en las tarjetas de video, PyBullet comprime todas las distancias en un rango estricto de `0.0` a `1.0`.

- `0.0` significa que el objeto está pegado al *Near Plane* (0.1m).
- `1.0` significa que el objeto está en el *Far Plane* (100m) o más allá.

Además, esta compresión **no es lineal**. La mayoría de la precisión (los decimales) se gasta en los primeros metros cerca de la cámara, y los objetos lejanos se comprimen todos juntos cerca del valor `1.0`.

# Linealización: De Z-Buffer a Metros Reales

Para que nuestro robot sepa la distancia real, debemos "descomprimir" esos valores de `0.0 - 1.0` usando una fórmula matemática estándar que depende de nuestro `near_val` y `far_val`.

La fórmula es:

$$Z = \frac{far \cdot near}{far - (far - near) \cdot z_{buffer}}$$

**En Python con Numpy, esto se aplica a toda la matriz a la vez:**

```
# depth_buffer contiene los datos del Indice 3 (re-
    dimensionados)

distancia_metros = far_val * near_val / (far_val - (far_val
    - near_val) * depth_buffer)

# Ahora, si queremos saber a que distancia esta el objeto
# exactamente en el centro de la pantalla:
distancia_frente = distancia_metros[alto//2, ancho//2]
print(distancia_frente, "metros")
```

# Visualizando la Profundidad en OpenCV

Si intentamos pasar la matriz `distancia_metros` directamente a `cv2.imshow`, OpenCV colapsará porque no sabe cómo dibujar números como "3.45 metros" en la pantalla. OpenCV necesita píxeles de 0 a 255 (`uint8`).

Para visualizarlo de forma sencilla, tomamos nuestro `depth_buffer` original (que ya va de 0 a 1), lo multiplicamos por 255 y le forzamos el tipo de dato:

```
import cv2

# Escalamos de (0.0-1.0) a (0-255)
profundidad_visual = (depth_buffer * 255).astype(np.uint8)

cv2.imshow("Profundidad", profundidad_visual)
# Los objetos cercanos se veran oscuros y el fondo blanco.
```